

离线版本测试流程

离线数据封装

首先拿到雷达采集的原始数据，然后进行封装，生成符合协议的离线数据文件。

本测试数据见 data/TEST2.json 。

具体步骤如下：

- 1. **准备原始数据：**本测试数据见 data/TEST2.json 。
- 2. **编写封装的脚本：**见 data/analysis_date.py ，
将原始数据按照协议进行封装。脚本需要处理数据头、数据体和数据尾的格式。

```
/* 雷达采样信息结构体 */
typedef struct {
    uint32_t frame_head;           /* 帧头(0x55AA55BB) */
    uint32_t frame_index;         /* 雷达推送帧索引 */
    uint16_t curr_pack;           /* 当前包序号 */
    uint16_t total_pack;         /* 总数据包数 */
    uint8_t data_type;            /* 数据帧类型(0:int8_t双极性ADC; 1:时域伪浮点; 2:1DFFT伪浮点; 3:2DFFT伪浮点; 4:CFAR; 5:点云) */
    uint8_t samp_ants;            /* 采样天线数 */
    uint16_t data_crc16;          /* 数据源CRC16 */
    uint16_t samp_points;         /* 采样点数据 */
    uint16_t samp_chirps;         /* 采样chirp数 */
    uint32_t data_bytes;          /* 采样数据源字节数 */
    uint8_t data[DATA_FIELD_BYTES]; /* 数据 */
    uint32_t frame_tail;          /* 帧尾(0x55CC55DD) */
} __attribute__((aligned(4))) sample_frame_t;

#define SAMPLE_FRAME_SIZE        sizeof(sample_frame_t)
/*****/
```

- 3. **运行封装脚本：**执行脚本，生成封装后的离线数据保存在同名.txt中，
- 4. **验证封装结果：**

✓ Python 封装完成。总字节数: 16412 (预期: 16412)

帧头: 0x55aa55bb, 帧尾: 0x55cc55dd

```
#define PACKED_FRAME_SIZE 16412
```

```
const uint8_t packed_frame_data[PACKED_FRAME_SIZE] = {
```

```
    0xBB, 0x55, 0xAA, 0x55, 0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x01, 0x00, 0x00, 0x02, 0xAD, 0xDE,  
    0x80, 0x00, 0x40, 0x00, 0x00, 0x40, 0x00, 0x00, 0xFF, 0x03, 0xF0, 0xE1, 0xD7, 0xED, 0xE9, 0xE6,
```

```
    0xF4, 0xDE, 0xDC, 0xE6, 0xEE, 0xE9, 0xE4, 0xE1, 0xF0, 0xF8, 0xEA, 0xEB, 0xDB, 0xDC, 0xFA, 0xFF,
```

```
    0xF6, 0xF1, 0xF2, 0xF5, 0x02, 0xFF, 0xDC, 0xE9, 0x0D, 0x13, 0x1D, 0xF6, 0xDC, 0xE5, 0xFE, 0x27,
```

```
    0x17, 0xF5, 0x00, 0xFD, 0xF2, 0x03, 0x14, 0xFE, 0x03, 0x02, 0xEE, 0xFB, 0x08, 0x00, 0x09, 0x04,
```

```
    0xEB, 0xDD, 0xF8, 0x07, 0x15, 0x04, 0xE7, 0xD9, 0xD7, 0xEB, 0x19, 0x05, 0xFF, 0xF2, 0xE9, 0xD8,
```

```
    0x07, 0x0C, 0xFE, 0x09, 0xF6, 0xEB, 0xEE, 0xEE, 0x11, 0x06, 0xF5, 0xF9, 0xF7, 0xED, 0x1E, 0x0B,
```

```
    0x01, 0xFF, 0xFB, 0xEB, 0x08, 0x19, 0x11, 0xF9, 0xFB, 0xF6, 0xF5, 0xFF, 0x07, 0x0D, 0xE0, 0xEA,
```

```
    0x0B, 0x02, 0x1D, 0x15, 0xE1, 0xE3, 0xF1, 0x08, 0x1D, 0x10, 0xEB, 0xE7, 0xE3, 0xF3, 0xF8, 0x00,
```

```
    0x00, 0xFC, 0x02, 0xF1, 0xDD, 0xDA, 0xEF, 0xFF, 0x26, 0x1E, 0x16, 0x0C, 0xFA, 0x06, 0x15, 0x15,
```

```
    0x02, 0x0B, 0x07, 0xFF, 0x0D, 0x18, 0x09, 0xFF, 0x1C, 0x19, 0x02, 0x10, 0x0D, 0x0D, 0x15, 0x29,
```

```
    0x0B, 0x08, 0x0E, 0x23, 0x17, 0x16, 0x22, 0x0E, 0x1A, 0x26, 0x1E, 0x00, 0x05, 0x16, 0x1F, 0x2C,
```

```
    0x2E, 0x0A, 0xFC, 0x13, 0x0E, 0x08, 0x2F, 0x27, 0x05, 0x00, 0x18, 0x1A, 0x12, 0x18, 0x15, 0x09,
```

```
    0x18, 0x19, 0x0E, 0x1E, 0x1E, 0x05, 0xF8, 0x0B, 0x0F, 0x1E, 0x22, 0x23, 0x01, 0xFD, 0x08, 0x18,
```

```
    0x21, 0x25, 0x17, 0x00, 0x05, 0x16, 0x10, 0x1E, 0x26, 0x18, 0x0E, 0x14, 0x16, 0x2C, 0x2E, 0x17,
```

```
    0x0A, 0x05, 0x26, 0x1E, 0x10, 0x21, 0x0B, 0xF6, 0x08, 0x24, 0x14, 0x0C, 0x17, 0x00, 0x04, 0x0F,
```

```
    0x1E, 0x14, 0x07, 0x11, 0x0B, 0xFF, 0x1D, 0x1F, 0x1B, 0x0D, 0x08, 0xF0, 0xFA, 0x28, 0x1D, 0x16,
```

```
    0x06, 0xFE, 0x02, 0x11, 0x17, 0x05, 0x02, 0x15, 0xF9, 0x00, 0xF6, 0xE3, 0xE0, 0xE6, 0xEE, 0xE7,
```

```
...
```

```
    0x26, 0x0D, 0x0A, 0x14, 0x10, 0xFA, 0x11, 0x1A, 0x17, 0x12, 0x0C, 0xF6, 0xFB, 0x23, 0x1F, 0x20,
```

```
    0x05, 0xFD, 0x07, 0x11, 0x17, 0x01, 0x05, 0x18, 0xDD, 0x55, 0xCC, 0x55,
```

```
};
```

5. 数据去向：将测试结果需要复制到 Tests/test_frame_data_dismantle.c 文件数组。

离线数据拆包测试

1. 编译测试代码：确保 CMakeLists.txt 中包含了 Tests/C 目录，然后运行 CMake 和编译命令。
2. 运行测试程序：执行生成的测试可执行文件 test_frame_dismantle。
3. 查看输出结果：程序会输出拆包后的数据，验证其是否与预期一致同时会打印前几个在终端可以看一下。

✅ frame_data_dismantle 成功解析数据。

Ant 0 / Chirp 0 / Point 0 (Raw: 255, Parsed: -1)

Ant 0 / Chirp 0 / Point 1 (Raw: 3, Parsed: 3)

Ant 1 / Chirp 63 / Point 127 (Raw Index 16383, Parsed: 24)

Ant 0 / Chirp 0-7 / Point 0 (验证重排):

Chirp 0: -1

Chirp 1: 38

Chirp 2: -7

Chirp 3: 30

Chirp 4: -6

Chirp 5: 30

Chirp 6: -9

Chirp 7: 31